

**Sistemas Móveis e Aplicações**

Licenciatura em Engenharia Informática

## **Relatório do Trabalho Prático**

SpaceHub



**UNIVERSIDADE DE ÉVORA**

**Miguel Pombeiro, 57829 | Miguel Rocha, 58501**

Departamento de Informática  
Universidade de Évora  
Maio 2026

# Índice

|          |                                                                   |          |
|----------|-------------------------------------------------------------------|----------|
| <b>1</b> | <b>Descrição da Aplicação</b>                                     | <b>2</b> |
| <b>2</b> | <b>Conceitos Lecionados e o Desenvolvimento</b>                   | <b>2</b> |
| 2.1      | Unidade 1: <i>Your first Android app</i> . . . . .                | 2        |
| 2.2      | Unidade 2: <i>Building app UI</i> . . . . .                       | 3        |
| 2.3      | Unidade 3: <i>Display lists and use Material Design</i> . . . . . | 3        |
| 2.4      | Unidade 4: <i>Navigation and app architecture</i> . . . . .       | 4        |
| 2.5      | Unidade 5: <i>Connect to the internet</i> . . . . .               | 5        |
| <b>3</b> | <b>Obstáculos Encontrados</b>                                     | <b>5</b> |
| <b>4</b> | <b>Objetivos Atingidos</b>                                        | <b>7</b> |

# 1 Descrição da Aplicação

## SpaceHub – o espaço, todos os dias, no seu bolso.

A SpaceHub é uma aplicação Android que aproxima o utilizador do universo, através de imagens, informações e conteúdos reais da NASA. Foi pensada para curiosos, entusiastas de astronomia e qualquer pessoa que queira explorar o espaço de forma acessível, sem ter de procurar informação dispersa em diferentes plataformas.

Através desta aplicação, é possível consultar a *Astronomy Picture of the Day* (APOD) do próprio dia, ou de dias anteriores. Também permite pesquisar e filtrar as imagens da própria NASA. Para além disso, é possível visualizar uma animação interativa construída a partir de fotos reais da Terra, captadas pela câmara EPIC (*Earth Polychromatic Imaging Camera*) da missão *DSCOVR*.

## 2 Conceitos Lecionados e o Desenvolvimento

Ao longo do semestre, os conceitos foram lecionados seguindo as várias (até à Unidade 5) unidades do curso "Android Basics with Compose" do *Google for Developers*. De forma a facilitar a compreensão do *mapping* no relatório, foi feita uma separação por unidade, sendo que cada uma irá conter os principais conceitos nela lecionados e utilizados no desenvolvimento da aplicação.

### 2.1 Unidade 1: *Your first Android app*

Esta unidade introduziu os fundamentos iniciais de Kotlin, a configuração inicial do Android Studio e a criação de uma primeira aplicação.

Relativamente aos fundamentos de Kotlin lecionados, estes foram bastante básicos, como funções e variáveis, estando a ser usados em diversos sítios, já que são fundamentais para a aplicação.

Alguns dos outros conceitos aprendidos aqui foram aplicados, desde a estrutura inicial do projeto, como, por exemplo, na *MainActivity*, onde a aplicação é iniciada através de `setContent()`, na criação dos vários ecrãs da aplicação, com funções *composable* reutilizáveis, que contêm elementos *composable* como `Box`, `Text`, `Column`, `Row`, bem como a utilização de *Modifiers* na maioria destas funções. Para cada um dos nossos ecrãs também realizamos um *Preview*, que nos ajudou a testar e aperfeiçoar a UI. Esta unidade também introduziu imagens (*Image*), que foram utilizadas, por exemplo, para mostrar o logótipo da aplicação nas barras de navegação.

Para além do que já foi referido, existem muitos mais conceitos aqui lecionados que estão implementados, como o *alignment* de *composables* e texto, *padding*, bem como

alteração de opacidade de uma imagem e o uso de *strings* no *Resource Manager* (de modo a preparar a aplicação para ser possivelmente traduzida).

Relativamente ao IDE escolhido para realizar o trabalho, não foi o Android Studio, mas sim o IntelliJ IDEA, devido às suas boas funcionalidades para trabalhar em equipa.

## 2.2 Unidade 2: *Building app UI*

Na Unidade 2, foram aprofundados ainda mais os conceitos de Kotlin, abordando condicionais, a nulabilidade de variáveis, classes, objetos, interfaces, funções lambda e mesmo testes automatizados. Ao nível do desenvolvimento em Android, mais especificamente, foram abordados conceitos como interfaces interativas, botões e eventos de clicar, estados em Compose e *state hoisting*, `TextField` (e `OutlinedTextField`), `Switch`, `Icons` *callbacks*, entre outros. No projeto, os conteúdos referidos foram aplicados na implementação de interações como a pesquisa de imagens (que inclui a utilização de filtros de pesquisa), seleção de datas, alteração do aspeto dos ecrãs e dos conteúdos mostrados, botões de *retry*, de partilha (botão "Share"), de reprodução ou o de carregamento de mais resultados.

O conceito de estado foi bastante explorado, porque a maior parte dos ecrãs da aplicação dependem de dados que mudam ao longo do tempo, como a *query* de pesquisa, os filtros, o índice da imagem atual, o estado de carregamento das imagens, eventuais erros, entre outros. Desta forma foi necessária a utilização de `mutableState` para atualizar os *composables* e `rememberSaveable` por forma a garantir a persistência destes estados.

Para além disso, também foi aplicada a técnica de *state hoisting* para passar valores e *callbacks* entre *composables*, tentando manter os componentes visuais o mais reutilizáveis possível. Desta forma, a maior parte da lógica ficou integrada nos `ViewModel`.

## 2.3 Unidade 3: *Display lists and use Material Design*

A unidade 3 introduziu coleções, classes, enums em Kotlin, e no geral, listas, grids, temas, tipografia, cards, ícone da aplicação, animações simples e acessibilidade. As respostas das APIs são convertidas em modelos como `NasaImage`, `EpicImage` e `ApodDto`, e são manipuladas com operações sobre coleções, como `map`, `filter`, `mapNotNull` e seleção de elementos. A nossa biblioteca de imagens `ImageSearchScreen`, usa `LazyColumn` e `LazyVerticalGrid` para apresentar resultados em lista ou grelha. Para adicionar uma identidade visual única, criamos o nosso próprio tema, quer seja com cores e tipografia, que permite a troca de cores automaticamente conforme o estilo do sistema, bem como a implementação de um ícone da aplicação, neste caso realizado na plataforma *Figma*. Utilizamos a ferramenta *Image Asset Studio*, para adicionar quer seja esse mesmo logo para a navegação, quer seja os próprios ícones da aplicação.

Também foram lecionados conteúdos como uma *TopBar*, que foi realizada e melhorada conforme o que foi dado na unidade 4, e animações simples, por exemplo, no ecrã da *library*

de imagens onde temos um botão que expande a secção de pesquisa por filtros.

Por fim, foi abordado a acessibilidade da aplicação, que é um tópico muito importante que tivemos em conta no desenvolvimento da aplicação ao adicionar descrições de conteúdo em imagens e ícones.

## 2.4 Unidade 4: *Navigation and app architecture*

Nesta unidade, foram abordados temas como a arquitetura de uma aplicação, o ciclo de vida da *Activity*, o *ViewModel*, o *UI State*, *StateFlow*, navegação entre ecrãs, bem como a adaptação da aplicação a diferentes tipos de ecrã. No SpaceHub, estes conteúdos foram aplicados através de uma arquitetura separada por camadas, com:

- Ecrãs *Compose*, responsáveis pela interface.
- *ViewModels*, responsáveis pelo estado e por toda a lógica de apresentação ao utilizador.
- Um repositório, responsável pelo acesso aos dados.

O uso de *UI State* permite representar estados como *loading*, sucesso, erro, vazio, por forma a tornar a interface mais previsível, fácil de manter e com uma melhor experiência de utilização para o utilizador.

Tal como foi lecionado, a navegação foi implementada com recurso ao *NavHost*, rotas e ecrãs distintos, como *Home*, *APOD*, *EPIC*, *Library*. Além disso, o ecrã *Image Detail*, que mostra detalhes sobre um dos resultados de pesquisa, inclui suporte para voltar para trás, uma vez que se trata de um ecrã secundário.

A adaptação a ecrãs de diferentes dimensões foi feita com recurso a *WindowWidth-SizeClass* e ao redimensionamento de alguns componentes, como o conteúdo do ecrã *EPIC*, que é ajustado para não ocupar mais do que o espaço disponível. A forma de navegação é, também ela, ajustada, alternando entre *bottom navigation*, *navigation rail* e *permanent drawer*.

Tal como já foi mencionado, de forma a demonstrar a interação com aplicações externas, foram implementados botões "Share", que permitem partilhar texto sobre *Cards*, incluindo o título da imagem e um link, que podem ser partilhados através de email, Whatsapp, entre outros.

Por fim, foi demonstrada a utilização de Logs, com recurso às diferentes fases do ciclo de vida de uma *Activity*. Sempre que cada um dos métodos *onCreate*, *onStart*, *onResume*, *onRestart*, *onPause*, *onStop* e *onDestroy* são chamados, é gerado um log a indicar que o método foi chamado.

## 2.5 Unidade 5: *Connect to the internet*

A unidade 5, apesar de não ter sido lecionada, fez parte do desenvolvimento da aplicação, sendo esta uma das unidades mais importantes, já que é esta que nos permite ter todas as funcionalidades que implementámos. Para tal, decidiu-se utilizar as APIs da NASA, em concreto a APOD (*Astronomy Picture of the Day*), EPIC (*Earth Polychromatic Imaging Camera*) e *NASA Image and Video Library*. Sugerimos ir buscar uma chave de API no próprio site, embora tenha sido utilizada uma chave demo, como *fallback*.

Esta unidade focou-se na ligação à internet, *coroutines*, HTTP/REST, Retrofit, chamada de dados, repositórios, injeção manual de dependências e carregamento de imagens com Coil. A classe `DefaultAppContainer` configura os serviços Retrofit e cria o `NetworkNasaRepository`, que centraliza as chamadas às APIs e transforma os DTOs recebidos em modelos usados pela interface.

A conversão de JSON para objetos Kotlin é feita com *kotlinx.serialization*, onde utilizamos as anotações `@Serializable` e `@SerializedName`. As operações de rede são feitas de forma assíncrona através de funções *suspend*, evitando bloquear a interface.

Para apresentar imagens da internet, foram usados componentes como `AsyncImage` com Coil, tanto em cards como nos ecrãs de detalhe e EPIC. Além disso, foram implementados estados de carregamento, erro e ausência de resultados, melhorando a experiência final do utilizador.

De forma geral, o desenvolvimento do SpaceHub permitiu aplicar os conteúdos das unidades lecionadas, começando pelos fundamentos de Kotlin e Compose, evoluindo para interfaces interativas e listas com Material Design, e culminando numa aplicação com uma arquitetura organizada, navegação, adaptação a diferentes ecrãs e consumo de dados reais através da internet.

Tentámos implementar todos os conteúdos lecionados e que identificamos através de uma análise cuidada das várias unidades do curso "Android Basics with Compose". De forma a não tornar este relatório demasiado extenso (ou por lapso), não foram explorados neste relatório todos os conceitos implementados.

## 3 Obstáculos Encontrados

Durante o desenvolvimento da aplicação, encontrámos vários obstáculos, especialmente porque, como já foi verificado na secção 2, foi necessário combinar desenvolvimento Android com desenho de *interface* de utilizador, navegação e APIs externas. Uma das primeiras dificuldades encontrada foi estudar e compreender as APIs utilizadas na aplicação. Visto que a aplicação depende de serviços da NASA, foi necessário estudar a documentação dos mesmos e verificar os *endpoints* disponíveis, os seus parâmetros, bem como o formato das respostas. Este processo foi bastante importante, uma vez que cada

uma das *features* da aplicação (e.g. APOD, *NASA Image Library*, e *EPIC*) dependiam de estruturas de dados diferentes. A interpretação das respostas em JSON e a sua serialização para *data models* de Kotlin necessitou de cuidado, visto que dados incompletos ou imprevistos podem comprometer o funcionamento da aplicação.

A navegação foi outro desafio importante, uma vez que a aplicação contém deferentes ecrãs, como a *Home*, APOD, entre outros. Criar o fluxo de navegação entre ecrãs, foi uma tarefa nova, que exigiu algum planeamento, nomeadamente ao nível da informação que é necessário passar de um ecrã para outro. Por exemplo, quando um utilizador seleciona uma imagem do ecrã de pesquisa, a aplicação precisa de navegar para um ecrã que exhibe o detalhe, enquanto é necessário preservar a informação dessa imagem.

Como todos os ecrãs da aplicação dependem de informação carregada através da Internet, a aplicação precisa de ter em consideração diferentes estados: sucesso, a carregar, erros e resultados vazios. Assim, foi necessário fazer uma gestão de estados no *frontend*, para garantir que a informação mostrada ao utilizador está correta. Apesar de contribuírem para melhorar a experiência de utilização da aplicação, estes estados acrescentaram complexidade na implementação.

O fluxo geral de ações de utilização também representou um obstáculo uma vez que a aplicação inclui múltiplos pontos de interação com o utilizador (e.g. selecionar datas, aplicar filtros, selecionar tipos de imagem na funcionalidade EPIC, partilhar conteúdo). Cada uma destas ações necessita de atualizar a parte correta da interface, bem como acionar as funções/lógica correta.

Como é pretendido que a aplicação funcione em ecrãs com diferentes tamanhos, o posicionamento e alinhamento dos componentes também foi um desafio. A utilização dos *layouts* do Jetpack Compose, como *Column*, *LazyColumn* e *LazyVerticalGrid* necessitou de alguma prática e habituação. Alguns componentes necessitaram, também, de diferentes comportamentos consoante a largura do ecrã, o que implicou que o *layout* tivesse de ser ajustado para que a experiência de utilização ficasse semelhante tanto em ecrãs compactos como mais largos.

Inicialmente, a organização e estruturação do projeto também representou um obstáculo, uma vez que este se tratou da primeira aplicação Android criada de raiz que abrangeu tantas funcionalidades e conceitos. Foi necessário separar responsabilidade entre a camada de rede, a camada de repositórios, os modelos de dados, os componentes de UI, ecrãs e *view models*. Compreender onde cada função/método e ficheiro de código deveria estar localizado nem sempre foi fácil. Contudo, a utilização desta estrutura em camadas torna os projetos mais fáceis de manter, bem como ajuda a evitar colocar demasiada lógica dentro de componentes da UI.

A injeção de dependências também necessitou de alguma atenção da nossa parte, uma vez que a aplicação precisa que os vários serviços e o repositório estejam disponíveis para os *view models* e ecrãs. A sua organização num *App Container* ajudou a "centralizar" a

criação de objetos, tendo sido necessário compreender como as dependências devem ser partilhadas pelos vários componentes da aplicação.

Por fim, compreender a estrutura de um projeto Android foi uma parte significativa do processo de aprendizagem. O projeto inclui, para além de ficheiros de código, ficheiros de configuração de Gradle, o *Android Manifest*, diretórios de recursos, recursos *drawables*, recursos *string*, temas, entre outros. Compreender o seu propósito foi necessário para desenvolver a aplicação de acordo com as regras que um projeto Android deve seguir, para que seja executado de forma correta.

Estas dificuldades, que acabamos por conseguir superar, foram importantes porque nos ajudaram a compreender melhor o desenvolvimento de uma aplicação Android.

## 4 Objetivos Atingidos

O objetivo principal deste trabalho foi criar uma aplicação Android funcional que demonstrasse os conhecimentos e ferramentas lecionados nas aulas práticas, no nosso caso, dedicada à exploração dos conteúdos disponibilizados pela NASA. Este objetivo foi atingido com o desenvolvimento do SpaceHub, que foi construído principalmente em Kotlin com Jetpack Compose.

Durante o desenvolvimento da aplicação foram aplicados vários conceitos de Android, desde, a criação de *Composables*, navegar entre ecrãs, gerir estados e até mesmo carregar imagens e dados da internet através APIs REST, para demonstrar conhecimento das funcionalidades aprendidas.

Após uma análise das APIs no começo do projeto, os objetivos ficaram mais claros, levando assim à delimitação das funcionalidades mais importantes e engraçadas dentro dos possíveis. Sendo assim surgiram-nos outros objetivos como a comunicação com as APIs de modo a realizar as funcionalidades desejadas.

Desde o início tentámos sempre criar uma interface visualmente atrativa e com uma boa experiência de utilização para o utilizador. Desta forma, fomos obrigados a colocar-mo-nos na sua posição, desafiando assim as nossas capacidades. Para tal, foram feitos alguns protótipos "à mão" daquilo que poderiam ser os vários ecrãs da aplicação (Figura 1).

Por fim, outro objetivo nosso, também relacionado com a experiência do utilizador, foi deixar a aplicação responsiva para os diversos *layouts*, o que nos levou a implementar uma navegação customizada e adaptada para dispositivos de diversos tamanhos.

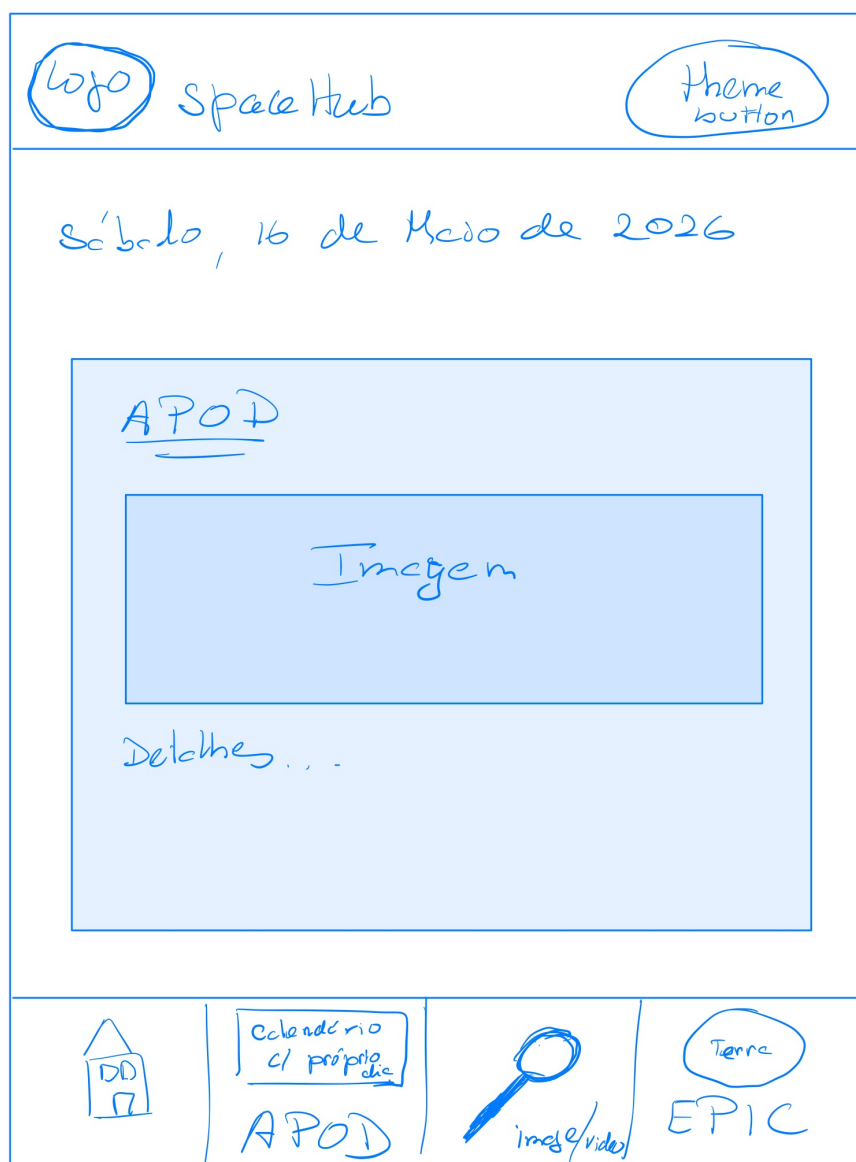


Figura 1: Protótipo à mão do ecrã *Home*.